

Университет ИТМО

Факультет программной инженерии и компьютерной техники

**Лабораторная работа №2**

по дисциплине «Системное программирование»

Выполнил:

Студент группы Р4114

Рыбин В. А.

Санкт-Петербург

2025

# Задание

Реализовать построение графа потока управления посредством анализа дерева разбора для набора входных файлов. Выполнить анализ собранной информации и сформировать набор файлов с графическим представлением для результатов анализа.

Порядок выполнения:

1 Описать структуры данных, необходимые для представления информации о наборе файлов, наборе

подпрограмм и графе потока управления, где:

а. Для каждой подпрограммы: имя и информация о сигнатуре, граф потока управления, имя

исходного файла с текстом подпрограммы.

б. Для каждого узла в графе потока управления, представляющего собой базовый блок алгоритма подпрограммы: целевые узлы для безусловного и условного перехода (по мере необходимости), дерево операций, ассоциированных с данным местом в алгоритме, представленном в исходном тексте подпрограммы

2 Реализовать модуль, формирующий граф потока управления на основе синтаксической структуры

текста подпрограмм для входных файлов

а. Программный интерфейс модуля принимает на вход коллекцию, описывающую набор

анализируемых файлов, для каждого файла – имя и соответствующее дерево разбора в виде

структуры данных, являющейся результатом работы модуля, созданного по заданию 1 (п. 3.б).

б. Результатом работы модуля является структура данных, разработанная в п. 1, содержащая

информацию о проанализированных подпрограммах и коллекция с информацией об ошибках

с.

Посредством обхода дерева разбора подпрограммы, сформировать для неё граф потока

управления, порождая его узлы и формируя между ними дуги в зависимости от синтаксической конструкции, представленной данным узлом дерева разбора: выражение,

ветвление, цикл, прерывание цикла, выход из подпрограммы – для всех синтаксических

конструкций по варианту (п. 2.б)

д. С каждым узлом графа потока управления связать дерево операций, в котором каждая

операция в составе текста программы представлена как совокупность вида операции и соответствующих операндов (см задание 1, пп. 2.d-g)

е. При возникновении логической ошибки в синтаксической структуре при обходе дерева разбора, сохранить в коллекции информацию об ошибке и её положении в исходном тексте

3 Реализовать тестовую программу для демонстрации работоспособности созданного модуля

а. Через аргументы командной строки программа должна принимать набор имён входных

файлов, имя выходной директории

б. Использовать модуль, разработанный в задании 1 для синтаксического анализа каждого

входного файла и формирования набора деревьев разбора

с.

Использовать модуль, разработанный в п. 2 для формирования графов потока управления

каждой подпрограммы, выявленной в синтаксической структуре текстов, содержащихся во

входных файлах

д. Для каждой обнаруженной подпрограммы вывести представление графа потока управления в

отдельный файл с именем "sourceName.functionName.ext" в выходной директории, по-умолчанию размещать выходные файлы в той же директории, что соответствующий входной

е. Для деревьев операций в графах потока управления всей совокупности подпрограмм

сформировать граф вызовов, описывающий отношения между ними в плане обращения их

друг к другу по именам и вывести его представление в дополнительный файл, по-умолчанию

размещаемый рядом с файлом, содержащим подпрограмму main.

ф.

Сообщения об ошибке должны выводиться тестовой программой (не модулем, отвечающим

за анализ!) в стандартный поток вывода ошибок

4 Результаты тестирования представить в виде отчета, в который включить:

а. В части 3 привести описание разработанных структур данных

б. В части 4 описать программный интерфейс и особенности реализации разработанного модуля

с.

В части 5 привести примеры исходных анализируемых текстов для всех синтаксических

конструкций разбираемого языка и соответствующие результаты разбора

# Используемые структуры

Для описания CFG графа была реализована следующая структура данных

```
typedef struct ControlGraphNode
{
    bool visited;
    size_t id;
    char *text;

    size_t need_to_unlock;
    size_t parent_amount;
    size_t parent_accum;

    bool connect_to_end;
    struct ControlGraphNode *end;

    struct ControlGraphNode *def;
    struct ControlGraphNode *cond;

    bool generate_asm;

    OpNode *operation_node;
} ControlGraphNode;
```

Для описания дерева операций была реализована следующая структура данных

```
typedef struct OperationTreeNode {
    OpNodeType type;
    char *argument;
    size_t children_amount;
    struct OperationTreeNode **children;
} OpNode;
```

## Примеры

Код:

```
function main()
    if 10 == 10 then
        puts("Some string\n");
    end if
end function
```

CFG:

```

<?xml version="1.0" encoding="utf-8"?>
<DirectedGraph
xmlns="http://schemas.microsoft.com/vs/2009/dgml"
Layout="Sugiyama" GraphDirection="TopToBottom">
<Nodes>
<Node Id="0" Label="main" />
<Node Id="1" Label="(10 == 10)" />
<Node Id="2" Label="Empty" />
<Node Id="3" Label="puts("Some string\n")" />
</Nodes>
<Links>
<Link Source="0" Target="1"/>
<Link Source="1" Target="2"/>
<Link Source="1" Target="3" StrokeDashArray="4,2" />
<Link Source="3" Target="2"/>
</Links>
</DirectedGraph>

```

Код:

```

function main()
    dim result, a, b as int
    result = a + b;
end function

```

CFG:

```

<?xml version="1.0" encoding="utf-8"?>
<DirectedGraph xmlns="http://schemas.microsoft.com/vs/2009/dgml"
Layout="Sugiyama" GraphDirection="TopToBottom">
<Nodes>
<Node Id="0" Label="main" />
<Node Id="1" Label="int result, a, b" />
<Node Id="2" Label="result = (a + b)" />
</Nodes>
<Links>
<Link Source="0" Target="1"/>
<Link Source="1" Target="2"/>
</Links>

```

</DirectedGraph>

## Вывод:

Было реализовано построение Control Flow Graph, также было изучено зачем оно вообще надо. Также было реализовано дерево операций, которое потребуется в дальнейшем для генерации asm-кода.